Production RAG in plain English

Written for a cloud / infra person, not a developer · a 10-minute read before an interview

1 Why basic RAG isn't enough

Basic RAG = embed the question → find the **most similar** chunks → give the top-K to the LLM. Fine in a demo, but at scale similar ≠ relevant. Zomato example: "order says delivered, never got it, refund?" The right record (ZH-002) ranked **3rd**. With **K = 2** the LLM never saw it.

🔧 A health check that only tests "port open": it looks healthy (similar) but the app is broken (not relevant).

7 ways it breaks → the fix

Similar mistaken for relevant	hybrid search + rerank
Missing evidence ("definitely refunded!")	grounded prompt + "I don't know"
Poor chunking (rule cut in half)	structure-aware / parent-child
No metadata filter (H&M policy to a Zomato user)	trusted tenant filter
No attribution	citations
No evaluation ("demo looked good")	labelled eval set
No failure handling	timeouts, fallbacks, fail closed

2 The production pipeline

❓ Question → 🪪 Identity → ✍️ Rewrite → 🔒 Access filter

🧠 Dense + abc BM25 → 🔀 RRF → 🏅 Rerank → 🤖 LLM → 🖱️ Cite

Example: dense **A,B,D,L** + BM25 **B,A,M,L** → RRF **A,B,L,D** → rerank **D,A,L,B** → top-2 **D,A** to the LLM. Without the rerank, D (the answer) is lost.

Router first

"What is 2 + 2?" Vector search **always** returns something. A router decides: RAG, a live API, analytics, or "not my job". 🔧 Ingress path rules.

Baseline first

Measure plain dense RAG on a fixed query set, then add **one** improvement at a time. 🔧 Load-test before tuning.

Chunking & retrieval words

Each term has a one-line meaning plus 🔧 an infra analogy

① Chunking

Fixed-size: cut every N characters (e.g. 120, overlap 20). Cuts words and rules in half.
🔧 Splitting a log file every 1 MB, mid-line.

Recursive: paragraph → sentence → word. The sensible default.

Structure-aware: one chunk per **section / FAQ item** (split on headings). Complete rules, easy citations.
🔧 One chunk per Kubernetes manifest, not per 50 lines.

Semantic: embed each sentence, cut where the **distance jumps** (0.12, 0.16, 0.71, 0.14 → cut). Costs more; the threshold depends on the model.
🔧 Alerting on a sudden spike, not a fixed value.

Parent-child: **search** small children (~300 chars), **return** the big parent (~1000).
🔧 grep the error line, then open the whole trace.

Contextual retrieval: an LLM adds "where this chunk comes from" **before** embedding. Anthropic: **−35%** failures (−49% with BM25).
🔧 Labels on every log line at ingest time.

Chunk metadata: document_id, section_path, version, updated_at, product, locale, status, **access scope**.

② Retrieval

Dense vector: 1536 numbers, **all filled**. Stores **meaning**. Finds paraphrases.

Sparse vector: 50,000 word slots, **almost all 0**. Stores **words**. Finds exact codes.
🔧 Search by employee ID vs by description.

BM25: keyword scoring: frequency (diminishing), **rare words count more** (IDF), length fairness. Knobs k1, b.
🔧 How Elasticsearch / log search ranks.

Inverted index: word → list of chunks containing it.
🔧 The index at the back of a book.

PAY-403 problem: dense search **misses error codes** it has never seen. BM25 catches them.

Hybrid search: dense + sparse **in parallel**, then merge. The production default.

RRF: Reciprocal Rank Fusion. `Σ 1/(60 + rank)`. Merges **ranks**, because cosine 0.77 and BM25 12.4 can't be added.
🔧 Merging two monitoring rankings by position, not raw values.

Query & safety words

Fixing the question side, and keeping it safe

③ Query intelligence

Query rewriting: "What if it has already passed?" → "What if my Zomato order's ETA has passed?" ZH-001 jumps to #1 (0.86).

Faithful rewrite: keeps IDs (ZM-123), adds **no facts**. Harmful = drops the ID or promises a refund.

Structured output: the LLM must return a fixed shape (`SearchQuery`, 3–400 chars).
🔧 A JSON schema validating an API response.

Self-query: the LLM extracts a query + **filters** (topic, product, locale) from the question.

Query routing: policy RAG vs **order API** (live status) vs **analytics** ("how many failed?") vs **action workflow** ("cancel my order").
🔧 Ingress rules: /docs → RAG, /orders → API.

Multi-query: several phrasings, search each, merge.

HyDE: write a fake answer first, search with it.

Step-back: ask a broader question first for background.

④ Safety & quality

Trusted identity: who is asking, checked **before** anything else (auth + RBAC).

Access filter: metadata `must` / `must_not`: tenant, team (sales / marketing / infra).
🔧 Namespaces + RBAC; a mandatory WHERE clause.

Never model-controlled: tenant_id comes from login, not from the LLM or user text. Self-query proposals are an **allowlist** check.

Grounded prompt: answer only from context.

Abstention: "I don't have that information, please contact support."

Citations: every claim → [ZH-002]. Makes answers auditable.

Recall@K vs Precision@K: found ÷ **all relevant** vs found ÷ **K**.

Eval set: fixed questions → expected chunks. Run on every change.
🔧 A CI regression suite.

Fail closed: retrieval timed out? **Refuse**, don't improvise.
🔧 A firewall that denies when rules can't load.

Before the interview

Read this page last. Say it out loud once.

"How do you make RAG production-grade?"

"First I build a **baseline** and an **eval set**. Then I fix it stage by stage. A **router** sends only document questions to RAG. **Identity** comes from the session and drives a **tenant/role filter**. Chunks are **structure-aware** with rich metadata, optionally **contextualised**. Retrieval is **hybrid**, dense + BM25 fused with **RRF**, then **reranked**. Follow-ups are **rewritten** into standalone queries. The LLM answers **only from context, with citations**, or abstains. Every external call has **timeouts and fallbacks**, and failures **fail closed**."

Quick answers

Similar vs relevant?

Close in meaning ≠ answers the question. The right chunk ranked #3 and K = 2 cut it.

Why hybrid?

Dense = paraphrases, BM25 = exact codes (PAY-403). Users mix both.

Why RRF?

Scores are on different scales, so fuse **ranks**: Σ 1/(60 + rank).

Best chunking?

Structure-aware by default. Semantic / parent-child only if eval proves it.

Contextual retrieval?

Prepend a source description per chunk. −35% / −49% failures, but one LLM call per chunk.

Query rewrite risk?

Dropping IDs or adding facts. Use the rewrite only to search; answer the original question.

Self-query security?

Allowlist the fields. Tenant/role always come from authentication.

"Live status of my order"?

Route it to the order API. Documents can't know live state.

Multi-tenant RAG?

tenant_id on every chunk + a mandatory server-side filter.

How do you know it improved?

Eval set before vs after: Recall@K, answer quality.

Numbers to remember

Dense vector	1536 floats
Sparse vocab	~50,000 slots
RRF k	60
Parent / child	1000 / 300 chars
Anthropic contextual	−35% / −49%

Watch outs

Semantic threshold	not universal
langchain-classic	fixes end Dec 2026
Rewrites	untrusted proposals
Next lecture	reranking

Chapter 03 ✓